Infraestrutura de software
Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

O enunciado pede a sincronização do embarque no metrô da METROREC: vagões e passageiros são threads, e o embarque precisa ser ordenado usando só mutex e variáveis de condição do pthreads. Entrego sync_car.c e sync_car.h com a struct station e as funções station_init, station_load_car, station_wait_for_car e station_on_board. Junto vão um Makefile que gera o executável sync_car, testes próprios (test_extra.c) e alvos de estresse e ThreadSanitizer. O teste oficial do professor passa com "Looks good!" em 500 de 500 execuções.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
struct station + station_init, station_load_car, station_wait_for_car, station_on_board	OK	make && ./sync_car → "Looks good!" (struct em sync_car.h, ver seção 4)
load_car só retorna com todos sentados e vagão cheio ou plataforma vazia	OK	O teste oficial acusa "returned early" ou "Many passengers"; make stress N=500 → 500/500 ok
wait_for_car só retorna com vagão na estação e assento livre	OK	make test-extra, caso (b): load_car(0) com 3 passageiros esperando libera 0
on_board avisa o vagão que o passageiro sentou	OK	Sem ele o vagão trava: "station_load_car failed" no commit f7b9322 (tentativa 4)
C, executável em Linux/Unix/macOS, só pthread_mutex_* e pthread_cond_*	OK	Testado em macOS 26.5.1 (arm64); grep -n pthread_ sync_car.c
Sem semáforos ou outras primitivas	OK	grep -n sem_ sync_car.c não retorna nada
No máximo um lock por struct station	OK	sync_car.h: único pthread_mutex_t lock

Requisito do enunciado	Status	Como testar / evidência
Um vagão por vez na estação, todos para o mesmo destino	OK	Premissa do enunciado; o teste cria um vagão de cada vez
Embarque simultâneo: vários wait_for_car retornam antes de qualquer on_board	OK	pthread_cond_broadcast(&car_arrived); wait_for_car não espera nenhum on_board, e boarding_passengers pode passar de 1
Sem busy-wait	OK	Toda espera em sync_car.c é pthread_cond_wait dentro de while; não há laço de polling
Makefile gera o executável sync_car	OK	make clean && make, sem warnings
README + Makefile com limpeza	OK	README.md; make clean remove executáveis e .o

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
make clean
make
```

Executar (exemplo):

```
./sync_car
```

4. Arquitetura (resumida)

Uma linha por arquivo, só o que é específico do seu projeto.

Arquivos e responsabilidades:

1. **sync_car.h → definição da struct station (lock, car_arrived, all_boarded, waiting_passengers, free_seats, boarding_passengers) e protótipos.**
2. **sync_car.c → implementação das quatro funções.**
3. **sync_car-runner.c → teste oficial do professor, incluído sem alterações (contém o main).**
4. **test_extra.c → testes próprios de station_load_car com count <= 0.**
5. **Makefile → alvos all, run, stress, tsan, test-extra e clean.**

Decisões técnicas:

1. **struct station completa em sync_car.h → o teste do professor faz #include "sync_car.h" e declara struct station station; na stack, o que exige o tipo completo → diverge do enunciado, que pedia a declaração em sync_car.c.**

2. **Duas variáveis de condição → passageiros e vagão esperam por eventos diferentes → broadcast em car_arrived libera vários passageiros de uma vez, e signal em all_boarded basta porque só o vagão espera ali.**
3. **Contador boarding_passengers (liberados que ainda não chamaram on_board) → o vagão só pode partir depois que todos sentarem → load_car espera enquanto (free_seats > 0 && waiting_passengers > 0) || boarding_passengers > 0.**
4. **free_seats = 0 antes de load_car retornar → um passageiro que chegue depois da partida não pode embarcar num vagão que já saiu → ele espera o próximo.**
5. **Todo pthread_cond_wait dentro de while → o broadcast acorda mais passageiros do que há assentos, e pode haver despertar espúrio → quem acorda sem assento volta a esperar.**
6. **on_board sinaliza com a mesma condição do while de load_car → evita acordar o vagão à toa → o vagão ainda reavalia a condição ao acordar.**
7. **Teste do professor sem alterações e -Wno-unused-parameter só para sync_car-runner.o → o runner gera um warning (foonão usado em alarm_handler) → sync_car.c continua compilando com -Wall -Wextra completos, sem warnings.**
8. **free_seats = count > 0 ? count : 0 → garante free_seats >= 0 mesmo com entrada inválida → é defensivo: hoje um valor negativo nunca fica visível (tentativa 7).**

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	1 mutex + 2 condições + 3 contadores	Passageiros esperam em car_arrived, o vagão em all_boarded; contadores waiting_passengers, free_seats, boarding_passengers	Não houve

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Estrutura inicial: header com campo provisório, stubs, Makefile (all, run, clean), README, .gitignore	Parcial	make falha: o teste do professor ainda não estava no diretório	28/10	commit 79f8997; log

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
2	S1	Campos da struct station + station_init; plano das três funções em comentários	OK		29/10	commit 476e3f1 (o commit gerado pela IA, 9dc7ecd, foi desfeito com git reset --soft e refeito por mim após revisar o diff)
3	S1	Adicionei o sync_car-runner.c do professor; Makefile e README ajustados ao nome com hífen	Falha esperada: "station_load_car returned early!"	Os stubs retornam na hora. Build com 1 warning no runner (foo não usado); 1º push falhou sem rede	29/10	commit 58401aa; print 1
4	S1	station_wait_for_car + station_load_car; -Wno-unused-parameter só no runner	Falha esperada: "station_load_car failed"	on_board ainda stub: boarding_passengers nunca volta a 0 e o vagão fica preso	03/11	commit f7b9322; log
5	S1	station_on_board com signal em all_boarded	OK		03/11	commit 9ee8591; print 2
6	S1	Alvos stress e tsan no Makefile	OK		03/11	commit e81b964; print 3; log
7	S1	Hipótese: count negativo deixaria passageiros embarcarem; criei test_extra.c para provar	OK		03/11	commit 5317601

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
8	S1	Endurecimento defensivo: free_seats nunca negativo	OK		03/11	commit 9e08580

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático (faz o trabalho pesado por você)

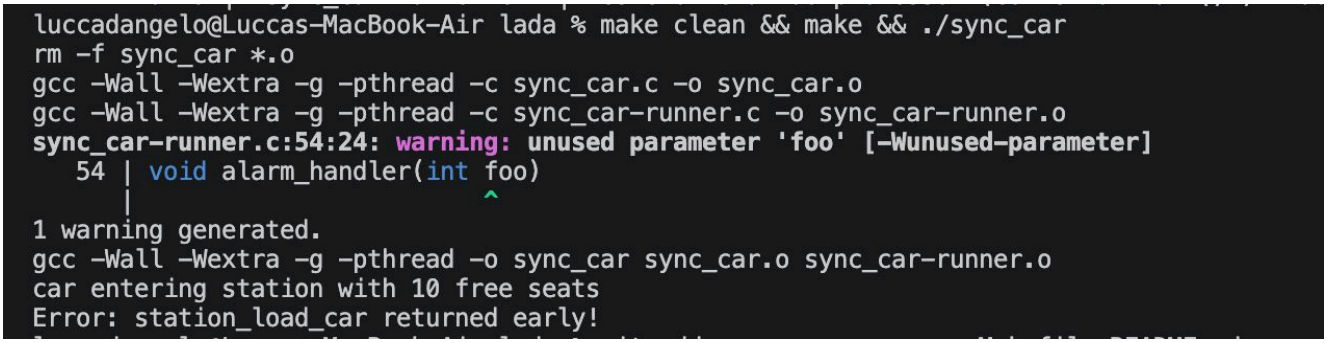
No início da sessão de trabalho, rode:

```
script -a evidencias.log
```

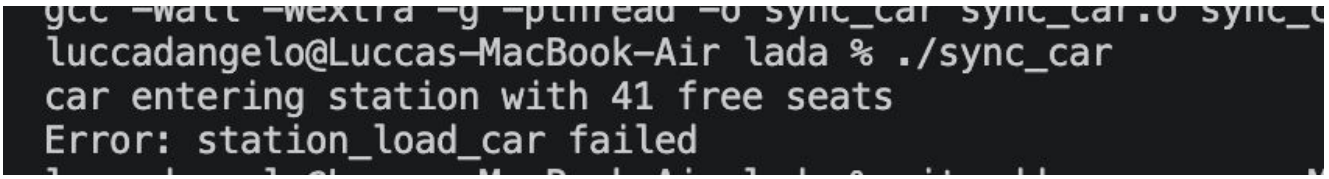
Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

6.2 Prints

- Erro principal: o momento do bug mais relevante da semana.
- Validação final: teste passando / saída correta.



1. Estado inicial: build + "station_load_car returned early!" com os stubs (tentativa 3).



2. Erro principal: "station_load_car failed", o vagão preso sem on_board (tentativa 4), recortado do evidencias.log.

```
luccadangelo@Luccas-MacBook-Air lada % ./sync_car
car entering station with 25 free seats
car departed station with 25 new passenger(s) (expected 25)
car entering station with 34 free seats
car departed station with 34 new passenger(s) (expected 34)
car entering station with 2 free seats
car departed station with 2 new passenger(s) (expected 2)
car entering station with 3 free seats
car departed station with 3 new passenger(s) (expected 3)
car entering station with 23 free seats
car departed station with 23 new passenger(s) (expected 23)
car entering station with 2 free seats
car departed station with 2 new passenger(s) (expected 2)
car entering station with 46 free seats
car departed station with 11 new passenger(s) (expected 11)
Looks good!
```

```
luccadangelo@Luccas-MacBook-Air lada % make stress N=500
X/500 ok
stress: 500/500 ok, 0 falhas
```

3. Validação final: make stress N=500 → "stress: 500/500 ok, 0 falhas" (tentativa 6).

7. Uso de IA

Obrigatório preencher mesmo se a resposta for "não utilizei".

Onde usei IA (2–3 linhas):

Usei o Claude (chat) para planejar a divisão em commits, fazer os comandos para testes, revisar as saídas e manter o diário deste relatório. Os commits e pushes foram feitos por mim.

Prompts principais (liste, sem história):

1. **Estrutura do projeto: header, stubs, Makefile e README.**
2. **Campos da struct station + station_init, com o plano das funções em comentários.**
3. **station_wait_for_car + station_load_car, e o warning do runner silenciado só para ele.**
4. **station_on_board + rodar o teste oficial 5 vezes.**
5. **Alvos stress e tsan, classificando cada aviso do TSan por arquivo.**
6. **test_extra.c com count <= 0, falhando antes da correção.**
7. **README final.**

O que validei manualmente e como:

- **Revisei cada diff antes de commitar e repeti make clean && make e os testes no terminal gravado.**
- **Desfiz o commit que o Claude Code fez sozinho (9dc7ecd) e refiz manualmente; a partir daí, todos os commits foram meus.**
- **Mantive o sync_car-runner.c do professor com o nome e o conteúdo originais, corrigindo o Makefile em vez do arquivo.**
- **A IA do chat sugeriu um bug com count negativo que não existia. O teste test_extra passou antes da correção (200/200) e a análise do lock refutou a hipótese. Registrei a mudança como defensiva e corrigi o comentário do código, que afirmava o bug.**
- **Conferi no README que cada resultado citado corresponde a uma execução real no log.**

8. Reflexão Final (5–8 linhas)

O que você melhoraria com mais tempo, principais decisões técnicas e erros que evitaria na próxima. Tudo junto, sem subseções separadas.

Reflexão:

A decisão central foi separar quem foi liberado (boarding_passengers) de quem ainda espera (waiting_passengers): sem isso, o vagão não sabe quando pode partir. Implementar função por função, rodando o teste do professor a cada etapa, deixou claro o papel de cada parte, porque cada falha intermediária tinha uma causa conhecida. O maior erro de processo foi o cronograma: fiquei de 30/10 a 02/11 sem trabalhar no projeto e comprimi a implementação em um dia. Também aceitei como bug uma hipótese da IA que eu não tinha testado (o count negativo); escrever o teste antes da correção foi o que mostrou o erro. Na próxima vez, testo em Linux além do macOS e salvo os prints no momento em que cada resultado aparece, em vez de recuperá-los do log depois.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	Sim
Rodei os testes e evidencias.log foi gerado	Sim
Tenho prints obrigatórios	Sim
Testei pelo menos um caso limite e um caso inválido	Sim
Preenchi a seção de uso de IA	Sim
Revisei o relatório e removi frases genéricas / vazias	Sim

10. Se eu tivesse mais 2 horas

Aplique as instruções abaixo de forma objetiva e verificável, evitando texto genérico.

Escreva 5-10 linhas sobre o que você melhoraria (robustez, testes, warnings, performance, organização).

1. Rodaria make stress e make tsan em Linux, com gcc, e usaria o Helgrind do Valgrind, que não está disponível no macOS arm64.
2. Faria uma cópia do runner só para o TSan, com pthread_join e variáveis atômicas, para ter uma execução com zero avisos, mantendo o original intacto para a avaliação.
3. Verificaria o retorno de pthread_mutex_lock, pthread_cond_wait e das funções de init, que hoje são ignorados.
4. Adicionaria uma station_destroy com pthread_mutex_destroy e pthread_cond_destroy.
5. Ampliaria o test_extra com cenários que o teste oficial não cobre: passageiros chegando durante o embarque e vários vagões seguidos com 1 assento.
6. Rodaria o stress com N=5000 para estimar melhor a probabilidade de falha intermitente.

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.